

# zeROS开发手册

## 1. 文档概要

本文档主要介绍zeROS的开发环境搭建、zeROS接口使用方法及zeROS软件开发中的注意事项。

### 1.1 名词解释

zeROS: 是基于zeromq和protobuf实现的一个用于机器人的消息中间件，提供了类似于ROS的API。

## 2. 开发环境

### 2.1 工程源码

源码仓库地址：[git@192.168.10.251:sweep\\_robot/P110/robot\\_main.git](https://github.com:git@192.168.10.251:sweep_robot/P110/robot_main.git)

Shell

```
1 $ git clone git@192.168.10.251:sweep_robot/P110/robot_main.git
2 $ cd robot_main
3 $ git submodule init
4 $ git submodule update --remote
5 $ git submodule foreach git checkout develop
6 $ git submodule foreach git pull origin develop
7 $ git checkout develop
```

| 主文件夹 workspace workspace-ros robot_main |  |      |         | Q | :: | ≡ | ⌵ | ⌵ | ⌵ |
|-----------------------------------------|--|------|---------|---|----|---|---|---|---|
| 名称                                      |  | 大小   | 已修改     |   |    |   |   |   |   |
| common                                  |  | 5 项  | 7月 5 日  |   |    |   |   |   |   |
| deploy                                  |  | 9 项  | 五       |   |    |   |   |   |   |
| docs                                    |  | 2 项  | 5月 26 日 |   |    |   |   |   |   |
| download                                |  | 14 项 | 3月 28 日 |   |    |   |   |   |   |
| environ                                 |  | 4 项  | 3月 23 日 |   |    |   |   |   |   |
| include                                 |  | 9 项  | 7月 5 日  |   |    |   |   |   |   |
| modules                                 |  | 12 项 | 7月 8 日  |   |    |   |   |   |   |
| porting                                 |  | 12 项 | 4月 28 日 |   |    |   |   |   |   |
| prebuilt                                |  | 5 项  | 6月 14 日 |   |    |   |   |   |   |
| third_party                             |  | 3 项  | 3月 26 日 |   |    |   |   |   |   |
| toolchain                               |  | 6 项  | 5日 5 日  |   |    |   |   |   |   |

- common: 公共库
- deploy: 发布目录
- docs: 文档目录
- download: 下载的开源代码目录
- environ: 编译及运行环境目录，包括预安装文件、根文件系统、交叉编译工具链
- include: 公共头文件
- modules: 子模块目录
- porting: 开源库移植目录
- prebuilt: 子模块目标发布目录
- third\_party: 第三方库目标发布目录
- toolchain: 交叉编译工具链目录
- utility: 工具目录

## 2.2 环境配置

2.2.1 首先请确保toolchain/tina-mr813v1.0.tar.gz 已经解压到toolchain目录下。如果没有，请解压(使用sudo):

## Shell

```
1 $cd toolchain
2 $cat tina-mr813v1.0.tar.gz.* | sudo tar zx
```

### 2.2.2 选择目标平台

## Shell

```
1 $source environ/envsetup.sh
2 $zbuild_setup
3 -----
4 please select configuration:
5   0 --> linux-x64-host
6   1 --> tina-mr813-p110
7   2 --> windows-x86_64
8 -----
9 which would you like? Default configuration: linux-x64-host
10 0
11 -----
12 build release version(default:no)? (no/yes)
13 no
14 Loading configuration for "linux-x64-host"
15 ZBUILD_PLATFORM : linux-x64
16 ZBUILD_PROJECT  : host
17 ZBUILD_ARCH     : x86_64
18 ZBUILD_SYSROOT  :
19 ZBUILD_TOOLCHAIN:
```

### 2.2.3 编译工程

## Shell

```
1 $mkdir build-x64-debug
2 $cd build-x64-debug
3 $cmake ..
4 $make install
```

### 2.2.4 其他平台

如果我们选择编译ARM平台,如mr813-p110,可以创建一个新的编译目录进行编译,编译完成后会自动安装到对应的deploy目录下。

## Shell

```
1 $mkdir build-p110-debug
2 $cd build-p110-debug
3 $cmake ..
4 $make install
5 $cd ..
6 $ls deploy
7 linux-x64-host-debug tina-mr813-p110
```

## 2.3 代码开发

### 2.3.1 使用zeros\_create\_pkg命令创建一个package，节点名称为demo\_zeros。

## Shell

```
1 $zeros_create_pkg src/demos/demo_zeros demo_zeros
2 $vim src/demos/demo_zeros/src/main.cpp
```

### 2.3.2 开始编写第一个zeros节点。

## C++

```
1 #include <zemb/BaseType.h>
2 #include <zemb/ArgUtil.h>
3 #include <zemb/Tracer.h>
4 #include <zeros/zeros.h>
5
6 using namespace zemb;
7 using namespace zeros;
8
9 int main(int argc, char** argv)
10 {
11     ArgOption option;
12     if(!option.parseArgs(argc, argv))
13     {
14         return RC_ERROR;
15     }
16     if (option.hasOption("h")>=0 ||
17         option.hasOption("help")>=0)
18     {
19         PRINT_YELLOW("%s [-h | --help] [-d level]", argv[0]);
20         return RC_OK;
21     }
22     zeros::init(argc, argv, "zeros_node");
23     if (option.hasOption("d")>0)
```

```

24     {
25         int level = std::stoi(option.getValue("d")[0]);
26         zeros::setlog(level,true,0);
27     }
28     else
29     {
30         zeros::setlog	TRACE_LEVEL_INFO,false;
31     }
32
33     /* 创建一个发布节点, 以50HZ的频率发布消息 */
34     NodeHandle nh;
35     auto puber = nh.advertise("/tester");
36     Thread::msleep(200); //不要立即发布消息, socket建立需要一定的时间
37     zeros::Rate rate(50);
38     while(zeros::ok())
39     {
40         std::string msg="hello";
41         puber->publish(msg);
42         rate.sleep();
43     }
44     zeros::shutdown();
45     return RC_OK;
46 }

```

### 2.3.3 修改CMakeLists.txt

#### Makefile

```

1  # this file is generated by zeros_create_pkg , see more in prebuild/README.md
2
3  ## cmake_minimum_required(VERSION <min>[...<max>] [FATAL_ERROR])
4  cmake_minimum_required(VERSION 3.0)
5
6  ## project(<PROJECT-NAME> [<language-name>...]), generated two variable
   PROJECT_SOURCE_DIR,PROJECT_BINARY_DIR
7  project(demo_zeros)
8
9  ##add_compile_options(<option> ...)
10 #add_compile_options(-std=c++14)
11
12 ##add_compile_definitions(<definitions> ...)
13 #add_compile_definitions(USE_POSIX)
14
15 ## set(<variable> <value>... [PARENT_SCOPE]), variable:
   DESTDIR,LD_FLAGS,CC,C_FLAGS,CXX,CXX_FLAGS...
16 #set(DESTDIR)
17
18 ## file(GLOB xxx_SRCS "${SOURCE_ROOT}/src/[a-zA-Z]*.cpp")

```

```
19 #file(GLOB xxx_SRCS "${SOURCE_ROOT}/src/[a-zA-Z]*.cpp")
20
21 ## file(<COPY|INSTALL> <files>... DESTINATION <dir> [FILES_MATCHING] [[PATTERN
    <pattern> | REGEX <regex>] [EXCLUDE] [PERMISSIONS <permissions>...]] [...])
22 #file(COPY files DESTINATION ${PROJECT_OUTPUT_INC}/${PROJECT_NAME})
23
24 ## find_package(<PackageName> [version] [EXACT] [QUIET] [MODULE] [REQUIRED]
    [[COMPONENTS] [components...]] [OPTIONAL_COMPONENTS components...]
    [NO_POLICY_SCOPE])
25 #find_package(xxx REQUIRED COMPONENTS
26 #)
27
28 ##find_library (<VAR> name1 [path1 path2 ...])
29 #find_library()
30
31 ##include_directories([AFTER|BEFORE] [SYSTEM] dir1 [dir2 ...])
32 include_directories(
33     inc
34     ${OpenCV_INCS}
35 )
36
37 ## link_directories([AFTER|BEFORE] directory1 [directory2 ...]), it must be
    before add_executable
38 #link_directories()
39
40 ## add_library(<name> [STATIC | SHARED | MODULE] [EXCLUDE_FROM_ALL] [source1]
    [source2 ...])
41 #add_library(${PROJECT_NAME}
42 #)
43
44 ## add_executable(<name> [WIN32] [MACOSX_BUNDLE] [EXCLUDE_FROM_ALL] [source1]
    [source2 ...])
45 add_executable(${PROJECT_NAME}
46     src/main.cpp
47 )
48 ##add_dependencies(<target> [<target-dependency>]...)
49 #add_dependencies(${PROJECT_NAME}
50 #${${PROJECT_NAME}_EXPORTED_TARGETS} ${xxx_EXPORTED_TARGETS})
51
52 ##target_include_directories(<target> [SYSTEM] [BEFORE]
    <INTERFACE|PUBLIC|PRIVATE> [items1...] [<INTERFACE|PUBLIC|PRIVATE> [items2...]]
    [...])
53 #target_include_directories(${PROJECT_NAME})
54
55 ## target_link_directories(<target> [BEFORE] <INTERFACE|PUBLIC|PRIVATE>
    [items1...] [<INTERFACE|PUBLIC|PRIVATE> [items2...]] [...])
56 #target_link_directories(${PROJECT_NAME})
```

```

57
58 ## target_link_libraries(<target> ... <item>... ...)
59 target_link_libraries(${PROJECT_NAME}
60     ${OpenCV_LIBS}
61 )
62
63 ## install(TARGETS <target>... [...])
64 install(TARGETS ${PROJECT_NAME}
65     DESTINATION ${PROJECT_OUTPUT_BIN}
66 )
67
68 ## install({FILES | PROGRAMS} <file>... [...])
69 # install(FILES files...
70 #     DESTINATION dest
71 # )
72
73 ## install(DIRECTORY <dir>... [...])
74 # install(DIRECTORY dir
75 #     DESTINATION dest
76 #     USE_SOURCE_PERMISSIONS
77 #     FILES_MATCHING PATTERN "*"*.h"
78 #     PATTERN ".git" EXCLUDE
79 # )
80
81 ## add_subdirectory(source_dir [binary_dir] [EXCLUDE_FROM_ALL])
82 #add_subdirectory(test_case)

```

2.3.4 编译及运行，编译通过后目标将被安装在deploy目录下的linux-x64的目标文件夹。

#### Shell

```

1 $cd build-x64
2 $cmake ..
3 $make install
4 $cd ../deploy/linux-x64
5 $source install/setup.sh
6 $./install/bin/demo_zeros

```

## 3. zeROS接口

### 3.1 节点句柄(NodeHandle)

3.1.1 每个节点代表着一个进程，在进程中定义的任何节点句柄(NodeHandle)实际上都是同一个节点。

3.1.2 为保证消息通信的效率，在一个节点中对同一个话题只允许创建一个订阅者。

## 3.2 消息循环(Spinner)

3.2.1 消息循环的频率不建议设置得过高，消息循环频率设置过大会增大CPU负载。消息循环频率必须大于消息发布频率，否则可能会出现消息丢失的问题。

3.2.2 一个节点默认有一个消息循环，在同一个消息循环中的任何一个回调对消息循环的延迟都有影响。我们应尽量让每一次回调的执行时间保持最短。对于订阅者和服务的回调，如果处理消息的时间大于消息循环的间隔时间，我们最好是先将消息缓存到一个队列中，另起一个线程来处理接收到的消息。

3.2.3 我们可以手动创建消息循环,并将订阅者或者服务绑定到该消息循环中。每一个消息循环都是一个独立的线程，在一个节点中非必要情况下应尽量少的使用消息循环。

C++

```
1 void callback_12(const std::string& msg) {
2     auto recvTime = Time::fromEpoch().toMicroSec();
3     auto frame = MsgHelper::unserialize<std_msgs::Header>(msg);
4     uint32 index = frame->index();
5     auto delay = recvTime - frame->stamp();
6     TRACE_DBG("thread[%u] recv msg index:%u , delay: %llu us, size: %d",
7         Thread::threadID(), index, delay, frame->frame_id().size());
8 }
9
10 void callback_34(const std::string& msg) {
11     auto recvTime = Time::fromEpoch().toMicroSec();
12     auto frame = MsgHelper::unserialize<std_msgs::Header>(msg);
13     uint32 index = frame->index();
14     auto delay = recvTime - frame->stamp();
15     TRACE_DBG("thread[%u] recv msg index:%u , delay: %llu us, size: %d",
16         Thread::threadID(), index, delay, frame->frame_id().size());
17 }
18
19 int main(int argc, char** argv) {
20     zeros::init("demo_ros_suber");
21     zeros::setlog(TRACE_LEVEL_DBG, true);
22     NodeHandle nh;
23
24     // 创建新的Spinner进行订阅
25     Spinner spinner = nh.spinner(1000);
26     auto suber01 = nh.subscribe("/topic_1", 1, callback_12, spinner);
27     auto suber02 = nh.subscribe("/topic_2", 1, callback_12, spinner);
28     // 启动新创建的Spinner消息循环
29     spinner->start();
30
31     // 默认Spinner订阅
32     auto suber03 = nh.subscribe("/topic_3", 1, callback_34);
33     auto suber04 = nh.subscribe("/topic_4", 1, callback_34);
34     zeros::Rate rate(1000);
35     while (zeros::ok()) {
36         zeros::spinOnce();
37         rate.sleep();
38     }
39     spinner->stop();
40     zeros::shutdown();
41     return RC_OK;
42 }
```

### 3.3 发布订阅(Topic)

3.3.1 在发布订阅模式中，创建发布者后，zmq需要一定的时间才能建立好socket，此时不要立即发布消息，否则可能会丢失最开始的几条消息。一般建议在advertise后等待200ms再发布消息。

### 3.4 服务(Service)

### 3.5 动作(Action)

### 3.6 域空间(Domain)

域用于区分节点的运行空间，只有在同一个域空间的节点间才能通信。如果没有设置环境变量ZEROS\_DOMAIN的情况下，节点运行在默认域空间。当有多个工作空间同时运行时，每个工作空间可以指定不同的域，以隔离工作空间，防止相互影响。注意：域名长度不能超过16个字符。

## Shell

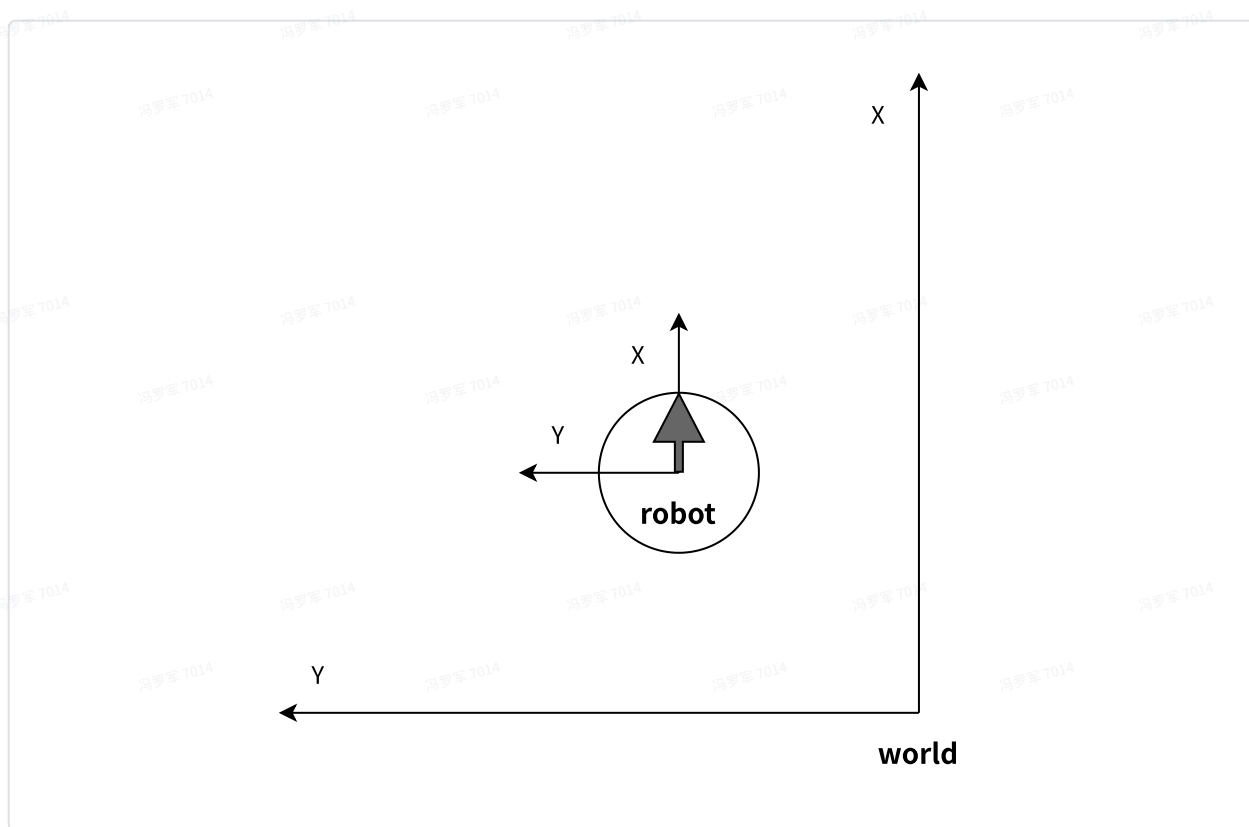
```
1 $export ZEROS_DOMAIN=domain_01
2 $source install/setup.sh
3
4         -----
5         |  __ \   /  __ \   /  ___ |
6  ____ | _ /   / _ \ | _ /   | | | | (___
7  | _ /   / _ \ | | \ | | \ | | | | ___ \
8  / ___ | \___ \ | | \ \ \___ \ | ___ \
```

#### 4. 统一坐标系

## 4.1 坐标系定义

### 4.1.1 机器人坐标系

机器人坐标系与世界坐标系一致，均使用右手坐标系。



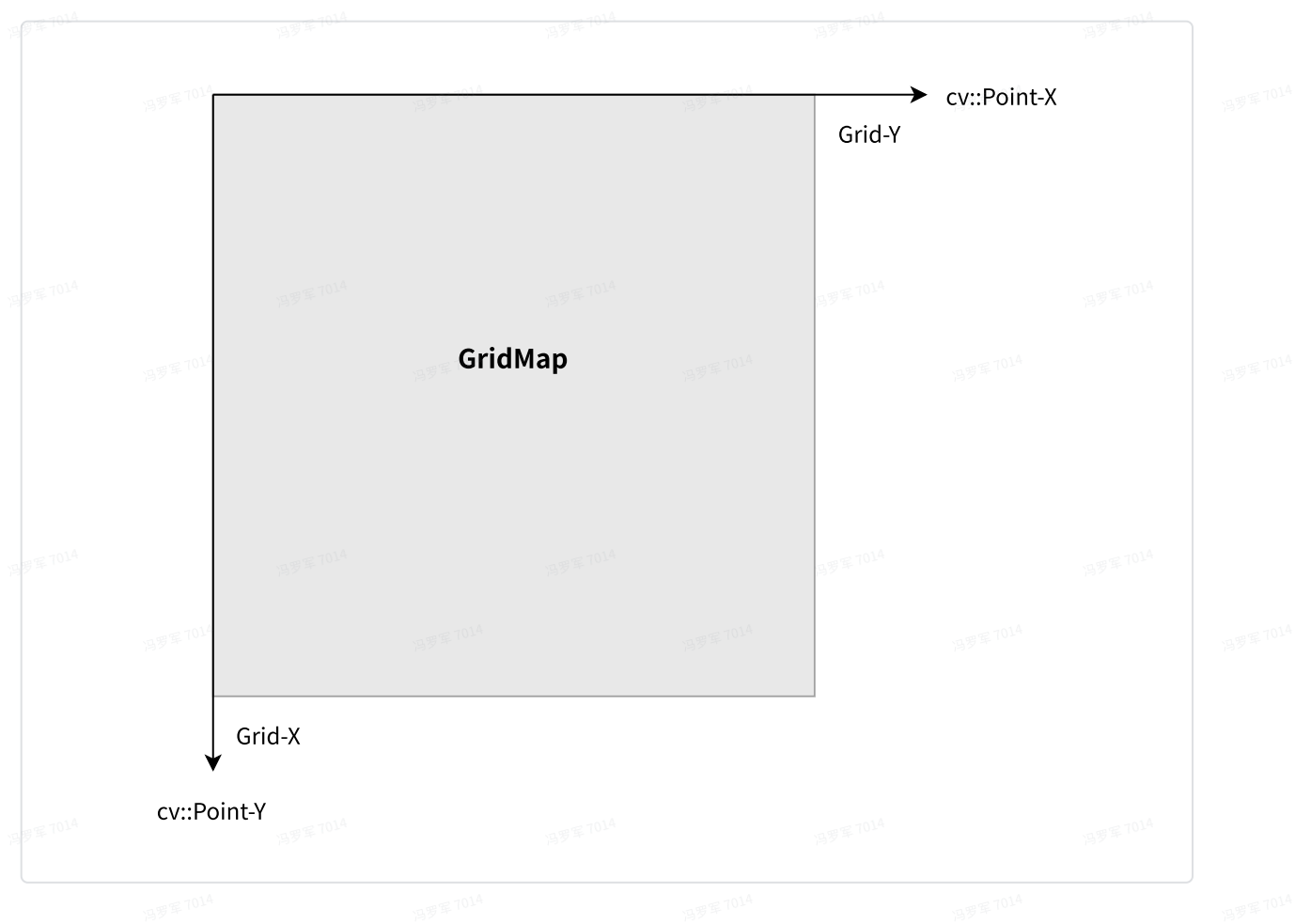
#### 4.1.2 栅格地图坐标系

为保持机器人坐标系与地图坐标系的统一(右手坐标系),我们规定栅格地图的X轴对应OpenCV图像的行(row), Y轴对应OpenCV图像的列(col)。这样当我们访问Grid(x,y)时实际对应cv::Mat(x,y)。

需要注意的一点是：在OpenCV中cv::Point(x,y)其X-Y轴与我们规定的地图坐标系轴正好相反，因此在使用cv::Point转换为Grid时x,y需要互换。即Grid(x,y)对应的是cv::Point(y,x)。

我们可以发现栅格地图坐标系实际也是右手坐标系，与机器人坐标系便统一起来了。

栅格地图默认大小为800\*800，地图分辨率为0.05m。



### 4.1.3 相机坐标系

在相机坐标系下图像上的每一个点的坐标为：

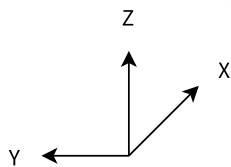
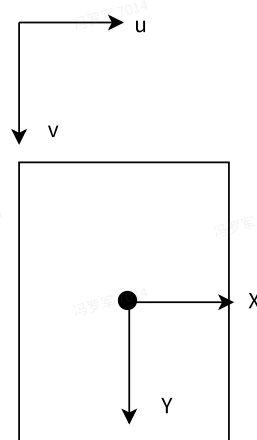
$$X = (u - C_x) * \text{depth} / F_x$$

$$Y = (v - C_y) * \text{depth} / F_y$$

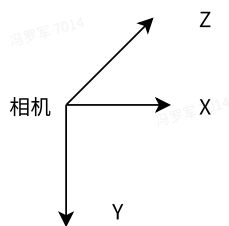
$$Z = \text{depth}$$

注:  $C_x$   $C_y$   $F_x$   $F_y$  为相机内参

像素坐标系



世界坐标系(机器人)



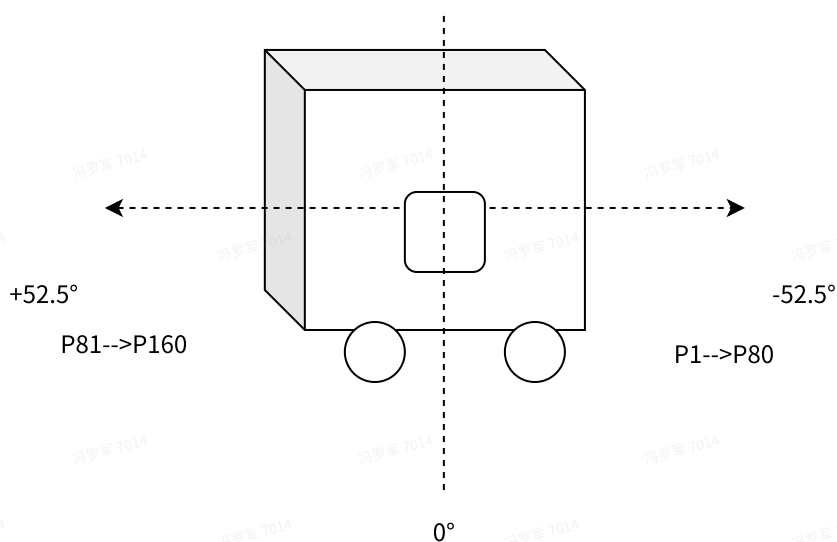
旋转矩阵(相机坐标系->世界坐标系):

$$R_z = \text{PI\_2}$$

$$R_y = -\text{PI}/2$$

$$R_x = 0$$

#### 4.1.4 线激光坐标系



## 4.2 坐标系转换

### 4.2.1 坐标转换

geometry包提供了获取旋转矩阵及点云坐标转换的接口，更多信息请查看geometry包源码：

[robot\\_main/src/comm\\_pkg/geometry](#)

C++

```
1  /**
2   * @brief 获取旋转矩阵(从当前坐标系到目标坐标系旋转,旋转顺序为Z-Y-X,内旋方式-绕自身轴旋
   转)
3   * @param yaw 绕Z轴旋转角度
4   * @param pitch 绕Y轴旋转角度
5   * @param roll 绕X轴旋转角度
6   * @return Matrix33<float>
7   * @note 统一使用右手坐标系,旋转角度为右手握住要旋转的轴,大拇指方向朝向旋转轴正方向,逆
   时针旋转为正
8   */
9  Matrix33<float> getRotationMatrixZYX(float yaw, float pitch, float roll);
10
11 /**
12  * @brief 坐标转换
13  * @param point 当前坐标系上的点
14  * @param rotation 旋转矩阵(从当前坐标系到目标坐标系旋转,由getRotationMatrix获取)
15  * @return Point3D point在目标坐标系下的坐标
16  */
17  Point3D transform(const Point3D& point, Matrix33<float>& rotation);
```